

Análisis de Algoritmos 2023/2024

Práctica 1

Antonio Moróño y Miguel Campo, 1202.

Código	Gráficas	Memoria	Total

1. Introducción

Antes de realizar la práctica, tuvimos que documentarnos acerca de la generación de números aleatorios haciendo uso del libro recomendado por el profesor (Numerical recipes in C: the art of scientific computing) y acerca de los algoritmos locales de ordenación que estamos estudiando en esta asignatura de análisis de algoritmos. Además, revisamos algunos aspectos fundamentales de la programación en C y el manejo del sistema operativo Ubuntu Linux para refrescar nuestros conocimientos después del verano. Tras los preparativos, estábamos listos para desarrollar la práctica.

2. Objetivos

Aquí indicamos el trabajo que vamos a realizar en cada apartado.

2.1 Generación de números aleatorios

- Familiarizarnos con la generación de números aleatorios usando la rutina `rand` de la librería `stdlib` y aprender cuál es la forma más correcta de generar números aleatorios.
- Introducción de la noción de histograma para comprobar el correcto funcionamiento de nuestro código.

2.2 Generación de permutaciones aleatorias

- Aprender un algoritmo para generar permutaciones formadas por números generados aleatoriamente.

2.3 Generación de lista de permutaciones aleatorias

- Crear una función que genera mediante la función `generate_perm` del ejercicio anterior `n_perms` permutaciones equiprobables de N elementos cada una que será útil para el resto de la práctica.

2.4 Algoritmo Select Sort

- Entender como funciona el método de ordenación **Select Sort** e implementarlo con un contador para el número de iteraciones.

2.5 Tiempos de ejecución

- Aprender cómo crear un programa que mide el tiempo de ejecución y el número de ejecuciones de la operación básica de un algoritmo para una serie de permutaciones aleatorias y guardar el resultado en un log.
- Escribir las funciones para determinar experimentalmente el tiempo medio de ejecución y el número medio, mejor y peor de veces que se ejecuta la Operación Básica del algoritmo por selección **Select Sort** sobre de una serie de permutaciones aleatorias.
- Escribir una función que visualice los datos obtenidos para compararlos con el análisis teórico

2.6 Select Sort Invertido

- Recrear estas pruebas para el mismo algoritmo pero que en vez de ordenar de menor a mayor, ordena de mayor a menos y comprobar las diferencias.

3. Herramientas y metodología

Hemos desarrollado la práctica en un entorno Linux, hemos usado los editores de código VSCode y Vim, la herramienta de gestión de memoria Valgrind y el compilador GCC. Además para las gráficas y visualización de datos también hemos usado GNUPlot y Excel.

Para todos los apartados hemos implementado control de errores y buena prácticas de programación.

3.1 Generación de números aleatorios

```
int random_num(int inf, int sup)
```

Implementación

Para construir la rutina hemos recurrido al libro recomendado (Numerical recipes in C: the art of scientific computing, capítulo 7). La solución que hemos encontrado es:

```
n = (int)((i) * rand() / (RAND_MAX + 1.0)) + inf;
```

Donde

- `i * rand()`: Esto multiplica el número aleatorio generado por `rand()` por el rango `i`.
- `/(RAND_MAX + 1.0)`: Esta división ajusta el número para que esté dentro del rango `[0, 1)`. Al dividir por `RAND_MAX + 1.0`, te aseguras de que el resultado sea un número en el intervalo `[0, 1)`. Luego, se suma `inf` para trasladar el rango al intervalo deseado, de `inf` a `sup`.

Pruebas realizadas

Para probar la función ejecutamos el comando: `./exercise1 -limInf 5 -limSup 14 -numN 1000000`

Que genera 1000000 de números aleatorios entre 5 y 14, y contamos cuantas veces aparece cada uno.

3.2 Generación de permutaciones aleatorias

```
int *generate_perm(int N)
```

Implementación

Simplemente hemos traducido el pseudocódigo a C.

Pruebas realizadas

Para probar la función ejecutamos el comando: `./exercise2 -size 5 -numP 1000000` Que genera 1000000 permutaciones aleatorias de tamaño 5 y con los datos obtenidos contamos cuantas veces aparece cada número en cada posición.

3.3 Generación de lista de permutaciones aleatorias

```
int **generate_permutations(int n_perms, int N)
```

Implementación

Reservamos memoria para un array de permutaciones y generamos **n_perms** de tamaño **N** usando la función del ejercicio anterior.

Pruebas realizadas

Para probar la función ejecutamos el comando `./exercise3 -size 5 -numP 1000000`

Que genera 1000000 permutaciones aleatorias de tamaño 5 y con los datos obtenidos contamos cuantas veces aparece cada número en cada posición.

3.4 Algoritmo Select Sort

```
int SelectSort(int *array, int ip, int iu)
```

Implementación

- Declaramos e inicializamos variables.
- Desde **ip** hasta **iu**, en cada iteración buscamos el mínimo entre **i** y **iu**.
- Intercambiamos el elemento en la posición **i** por el mínimo calculado
- Incrementamos la posición **i** y el número de iteraciones con cada iteración.
- Devolvemos el número de iteraciones (**ob**)

Pruebas realizadas

Para probar el algoritmo ejecutamos el comando: `./exercise4 -size 15` Que genera una permutacion aleatoria de tamaño 15 que es ordenada por el algoritmo.

3.5 Tiempos de ejecución

Implementación

```
short average_sorting_time(pfunc_sort metodo, int n_perms, int N, PTIME_AA  
ptime)
```

- Declaramos e inicializamos variables
- Generamos un array con **n_perms** permutaciones de tamaño **N** con la función `int **generate_permutations(int n_perms, int N)`
- Para cada permutación medimos el tiempo que "metodo" (en nuestra práctica Select Sort) tarda en ordenarlas, guardamos el máximo y el mínimo de tiempo que ha tardado en ordenar una permutación y contamos el número de iteraciones que hace en cada iteración.
- Liberamos la memoria de las permutaciones

```
short generate_sorting_times(pfunc_sort method, char *file, int num_min, int  
num_max, int incr, int n_perms)
```

- Reservamos memoria para un puntero a la struct **time_aa** teniendo en cuenta las variables de entrada.
- Desde **num_min** hasta **num_max** incrementando **incr** ejecutamos la función `short average_sorting_time(pfunc_sort metodo, int n_perms, int N, PTIME_AA ptime)`

- Guardamos los datos **ptime** en **file** con la función `short save_time_table(char *file, PTIME_AA ptime, int n_times)`
- Liberamos la memoria reservada.

```
short save_time_table(char *file, PTIME_AA ptime, int n_times)
```

- Abrimos **file** en modo escritura.
- Para el numero de filas **n_times** printeamos todos los datos en el puntero **ptime**.
- Cerramos el archivo en el que escribimos.

Pruebas realizadas

Para probar las funciones ejecutamos el comando: `./exercise5 -num_min 2000 -num_max 50000 -incr 2000 -numP 50 -outputFile exercise5.log`

Que genera 50 permutaciones desde tamaño 2000 hasta tamaño 50000 incrementando el tamaño de 2000 en 2000 en cada iteración y a continuación son ordenadas por Select Sort. Imprimimos en **exercise5.log** el tamaño de cada permutación, el tiempo que tarda en ordenarla, el numero de OB promedio, máximo y mínimo que hace al ordenar cada permutación.

3.6 Select Sort Invertido

```
int SelectSortInv(int *array, int ip, int iu)
```

Implementación

Igual que **Select Sort** pero en vez de recorrer la permutación desde **ip** hasta **iu** lo hacemos al revés.

Pruebas realizadas

Para probar el algoritmo ejecutamos el comando: `./exercise6 -size 15`

Que genera una permutacion aleatoria de tamaño 15 que es ordenada por el algoritmo.

4. Código fuente

4.1 Generación de números aleatorios

```

/*****
/* Function: random_num Date: 22-09-2023 */
/* Authors: Miguel Campo, Antonio Moroño */
/* */
/* Rutine that generates a random number */
/* between two given numbers */
/* */
/* Input: */
/* int inf: lower limit */
/* int sup: upper limit */
/* Output: */
/* int: random number */
*****/
int random_num(int inf, int sup)

```

```

{
    int n;
    double i = (double)sup - inf + 1;

    n = (int)((i)*rand() / (RAND_MAX + 1.0)) + inf;

    return n;
}

```

4.2 Generación de permutaciones aleatorias

```

/*****
/* Function: generate_perm Date: 22-09-2023 */
/* Authors: Miguel Campo, Antonio Moroño */
/*
/* Rutine that generates a random permutation */
/*
/* Input: */
/* int n: number of elements in the permutation */
/* Output: */
/* int *: pointer to integer array */
/* that contains the permutation */
/* or NULL in case of error */
*****/
int *generate_perm(int N)
{
    int i, *perm, aux, random;

    if (N < 1)
    {
        return NULL;
    }
    perm = (int *)malloc(sizeof(int) * N);

    if (perm == NULL)
    {
        return NULL;
    }

    for (i = 0; i < N; i++)
    {
        perm[i] = i + 1;
    }

    for (i = 0; i < N; i++)
    {
        aux = perm[i];
        random = random_num(i, N - 1);
        perm[i] = perm[random];
        perm[random] = aux;
    }
}

```

```

    return perm;
}

```

4.3 Generación de lista de permutaciones aleatorias

```

/*****
/* Function: generate_permutations Date: 29-09-2023*/
/* Authors: Miguel Campo, Antonio Moroño */
/* */
/* Function that generates n_perms random */
/* permutations with N elements */
/* */
/* Input: */
/* int n_perms: Number of permutations */
/* int N: Number of elements in each permutation */
/* Output: */
/* int**: Array of pointers to integer that point */
/* to each of the permutations */
/* NULL en case of error */
*****/
int **generate_permutations(int n_perms, int N)
{
    int **array, i;

    if (N <= 0 || n_perms <= 0)
    {
        return NULL;
    }

    array = (int **)malloc(sizeof(int *) * n_perms);
    if (!array)
    {
        return NULL;
    }

    for (i = 0; i < n_perms; i++)
    {
        array[i] = generate_perm(N);

        if (array[i] == NULL)
        {
            for (i--; i >= 0; i--)
            {
                free(array[i]);
            }
            free(array);
        }
    }
}

```

```

    return array;
}

```

4.4 Algoritmo Select Sort

```

/*****
/* Function: SelectSort    Date: 29-09-2023    */
/* Authors: Miguel Campo, Antonio Moroño      */
/* It implements the select sort algorithm      */
/*                                              */
/* Input:                                      */
/* array: elements to be sorted                */
/* ip: first index                            */
/* iu: last index                             */
/* Output:                                      */
/* Number of iterations                        */
*****/
int SelectSort(int *array, int ip, int iu)
{
    int i, mini, aux, ob = 0;
    i = ip;

    if (!array || ip < 0 || iu < ip)
    {
        return -1;
    }

    while (i < iu)
    {
        mini = min(array, i, iu);
        ob = ob + iu - i;
        aux = array[i];
        array[i] = array[mini];
        array[mini] = aux;
        i++;
    }

    return ob;
}

/*****
/* Function: min    Date: 29- 9-2023    */
/* Authors: Miguel Campo, Antonio Moroño */
/* Returns the lowest element from a given subtable */
/*                                              */
/* Input:                                      */
/* array: elements to be sorted                */
/* ip: first index                            */
/* iu: last index                             */
/* Output:                                      */
/* lowest element                             */
*****/

```



```

/*****/
int min(int *array, int ip, int iu)
{
    int j, min;
    min = ip;

    for (j = ip + 1; j <= iu; j++)
    {
        if (array[j] < array[min])
        {
            min = j;
        }
    }
    return min;
}

```

4.5 Tiempos de ejecución

```

/*****/
/* Function: average_sorting_time Date: 6-10-2023 */
/* Authors: Miguel Campo, Antonio Morono */
/* It stores in ptime data about the functioning */
/* of the metodo function */
/* */
/* Input: */
/* metodo: sorting method */
/* n_perms: number of permutations */
/* N: size of the permutations */
/* ptime: where the info will be stored */
/* Output: */
/* -1 if there is an error, 0 if it is OK */
/*****/
short average_sorting_time(pfunc_sort metodo, int n_perms, int N, PTIME_AA
ptime)
{
    clock_t ini, fin;
    int min = 0, max = 0, i, **perms, it = 0, itac = 0;
    double total_time = 0;

    if (!metodo || n_perms <= 0 || N <= 0 || !ptime)
    {
        return -1;
    }

    perms = generate_permutations(n_perms, N);

    if (!perms)
    {
        return -1;
    }
}

```

```

for (i = 0; i < n_perms; i++)
{
    if (!perms[i])
    {
        for (i = 0; i < n_perms; i++)
        {
            free(perms[i]);
        }
        free(perms);
        return -1;
    }
    ini = clock();
    itac = metodo(perms[i], 0, N - 1);
    it = it + itac;
    fin = clock();
    if (itac > max)
    {
        max = itac;
    }
    if (itac < min || min == 0)
    {
        min = itac;
    }
    total_time = total_time + (double)(fin - ini) / CLOCKS_PER_SEC;
}
ptime->average_ob = (double)it / n_perms;
ptime->max_ob = max;
ptime->min_ob = min;
ptime->N = N;
ptime->n_elems = n_perms;
ptime->time = (double)total_time / n_perms;

for (i = 0; i < n_perms; i++)
{
    free(perms[i]);
}
free(perms);
return 0;
}

/*****
/* Function: generate_sorting_times Date:20-10-2023*/
/* Authors: Miguel Campo, Antonio Moroño */
/* It creates data about the functioning of method */
/* */
/* Input: */
/* method: sorting method */
/* file: name of the file to store the info */
/* num_min: lowest size of permutations */
/* num_max: highest size of permutations */
/* incr: difference of size between iterations */
/* n_perms: number of permutations */
/* Output: */
/* -1 if error, 0 if not */

```

```

/*****/
short generate_sorting_times(pfnc_sort method, char *file, int num_min,
int num_max, int incr, int n_perms)
{
    int i, j = 0;
    PTIME_AA ptime;

    ptime = malloc(sizeof(TIME_AA) * ((num_max - num_min) / incr + 1));
    if (!ptime)
    {
        return -1;
    }

    for (i = num_min; i <= num_max; i = i + incr)
    {
        if (average_sorting_time(method, n_perms, i, &ptime[j]) == -1)
        {
            free(ptime);
            return -1;
        }
        j++;
    }

    if (save_time_table(file, ptime, (num_max - num_min) / incr) == -1)
    {
        free(ptime);
        return -1;
    }

    free(ptime);
    return 0;
}

/*****/
/* Function: save_time_table    Date: 6-10-2023    */
/* Authors: Miguel Campo, Antonio Moroño    */
/* It saves data from ptime in a file    */
/*    */
/* Input:    */
/* file: name of the file to store the info    */
/* ptime: where the info is taken from    */
/* n_times: number of rows in the file    */
/* Output:    */
/* -1 if error, 0 if not    */
/*****/
short save_time_table(char *file, PTIME_AA ptime, int n_times)
{
    int i;

    FILE *entrada;

    if (file == NULL || n_times < 1 || !ptime)
    {

```

```

    return ERR;
}

entrada = fopen(file, "w");
if (entrada == NULL)
{
    return ERR;
}

for (i = 0; i < n_times; i++)
{
    fprintf(entrada, "%d\t%f\t%f\t%d\t%d\n", ptime[i].N, ptime[i].time,
ptime[i].average_ob, ptime[i].max_ob, ptime[i].min_ob);
}

fclose(entrada);

return OK;
}

```

4.6 Select Sort Invertido

```

/*****
/* Function: SelectSortInv    Date: 29-09-2023    */
/* Authors: Miguel Campo, Antonio Moroño        */
/* It implements the inverted select sort algorithm*/
/*                                                */
/* Input:                                        */
/* array: elements to be sorted                */
/* ip: first index                             */
/* iu: last index                              */
/* Output:                                     */
/* number of iterations                        */
*****/
int SelectSortInv(int *array, int ip, int iu)
{
    int i, mini, aux, ob = 0;
    i = iu;

    if (!array || ip < 0 || iu < ip)
    {
        return -1;
    }
    while (i > ip)
    {
        mini = min(array, ip, i);
        ob = ob + i - ip;
        aux = array[i];
        array[i] = array[mini];
        array[mini] = aux;
        i--;
    }
}

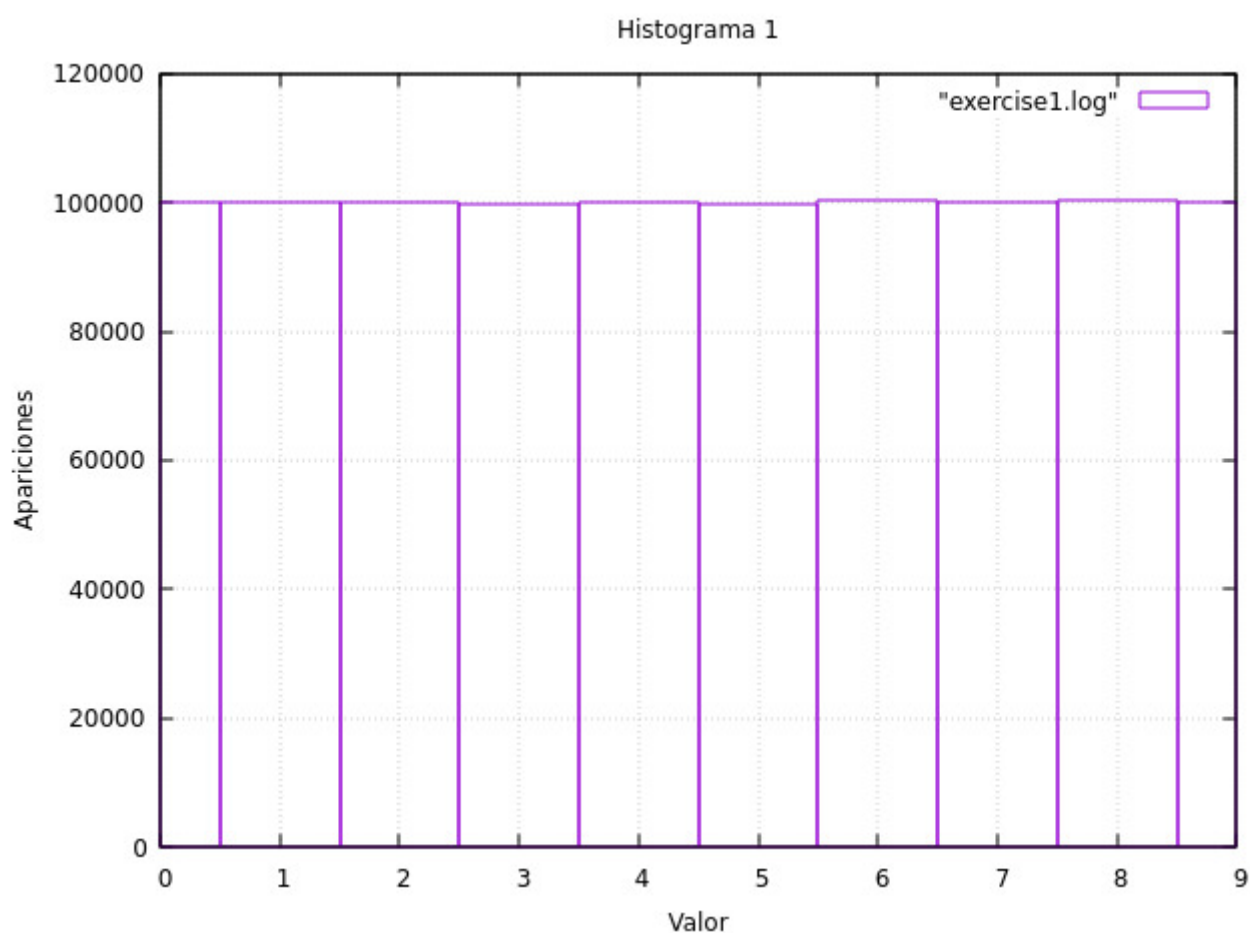
```

```
}  
  
    return ob;  
}
```

5. Resultados y Gráficas

5.1 Generación de números aleatorios

Comando ejecutado: `./exercise1 -limInf 5 -limSup 14 -numN 1000000`



Como podemos ver en el histograma se confirma la equiprobabilidad de los números generados. Hemos generado 1 millón de números entre 0 y 9, y cada uno ha aparecido aproximadamente 100.000 veces como esperábamos.

$$\frac{10^6}{10} = 10^5$$

5.2 Generación de permutaciones aleatorias

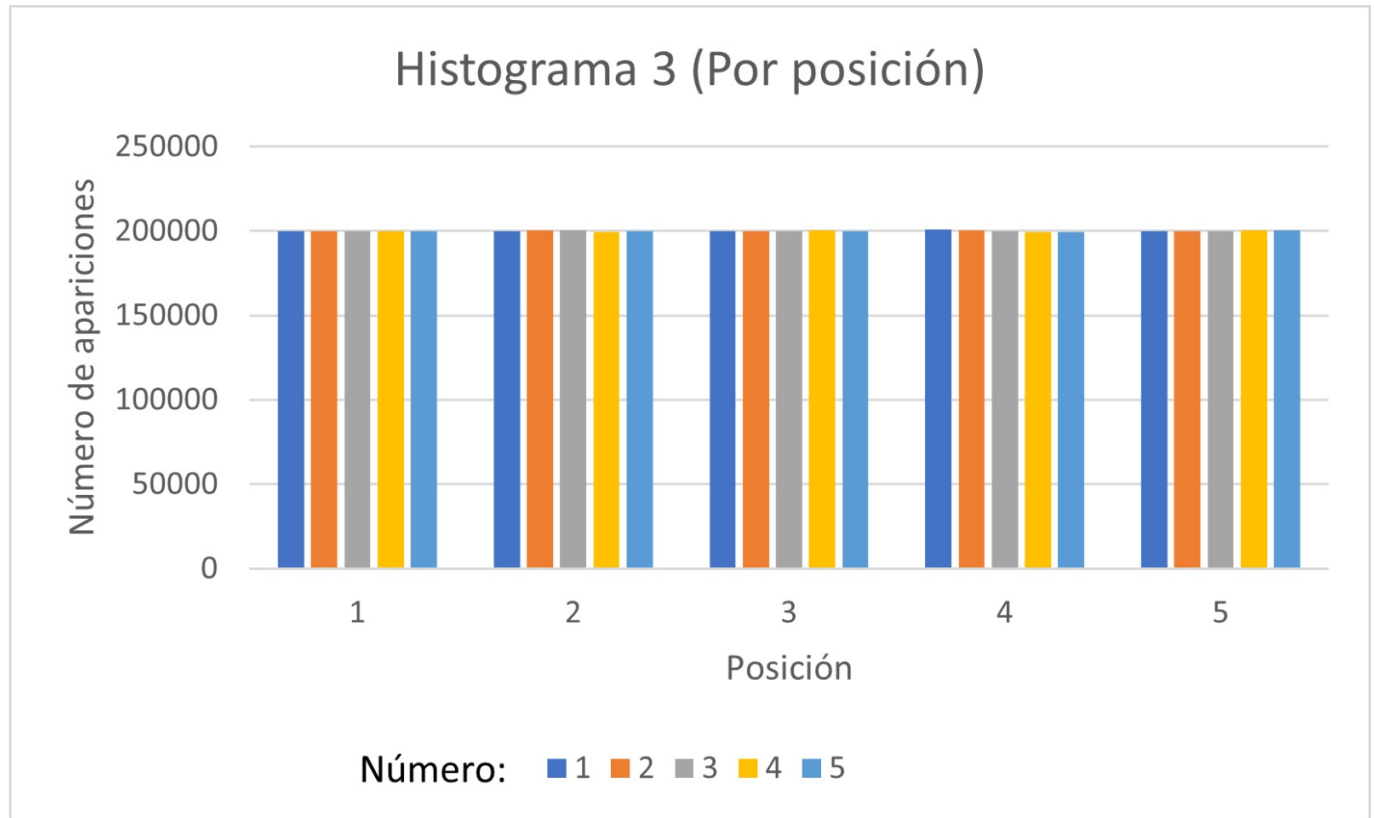
Comando ejecutado: `./exercise2 -size 5 -numP 1000000`

Teniendo en cuenta que las permutaciones generadas en este apartado están hechas con la función que acabamos de comprobar que es equiprobable es lógico pensar que las permutaciones generadas se rigen

por esta norma. A continuación en el siguiente apartado comprobaremos en profundidad este aspecto ya que ambos apartados están muy relacionados.

5.3 Generación de lista de permutaciones aleatorias

Comando ejecutado: `./exercise3 -size 5 -numP 1000000`



En este histograma cada grupo de columnas representa el número de veces que aparece cada número en esa posición. A simple vista podemos ver que cada número aparece el mismo número de veces en cada posición. Generamos 1 millón de permutaciones de tamaño 5 y vemos que cada número aparece 200.000 veces en cada posición como esperábamos.

$$\frac{10^6}{5} = 2 \cdot 10^5$$

5.4 Algoritmo Select Sort

Comando ejecutado: `./exercise4 -size 15`

Tras la ejecución podemos comprobar que efectivamente la lista de 15 elementos está ordeanda de menor a mayor como esperábamos.

5.5 Tiempos de ejecución

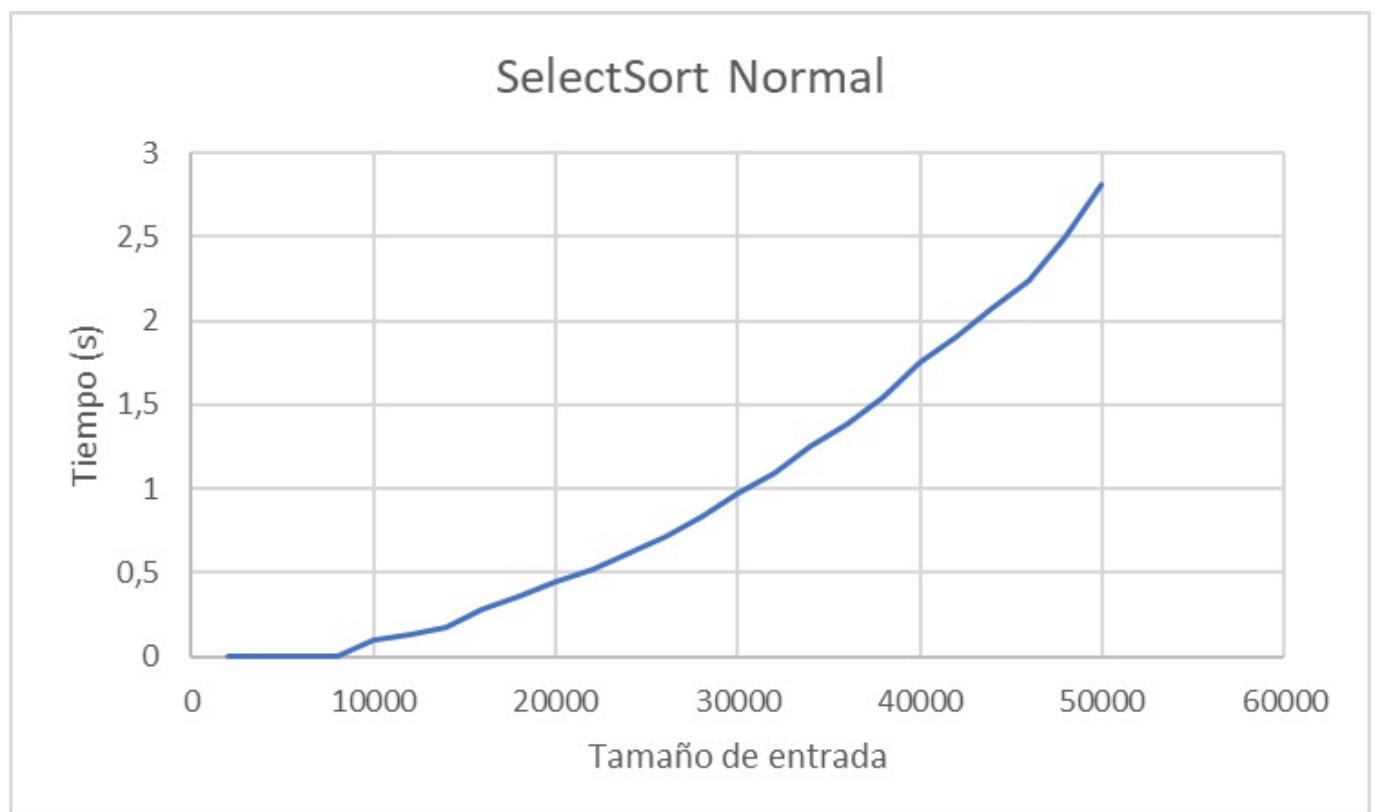
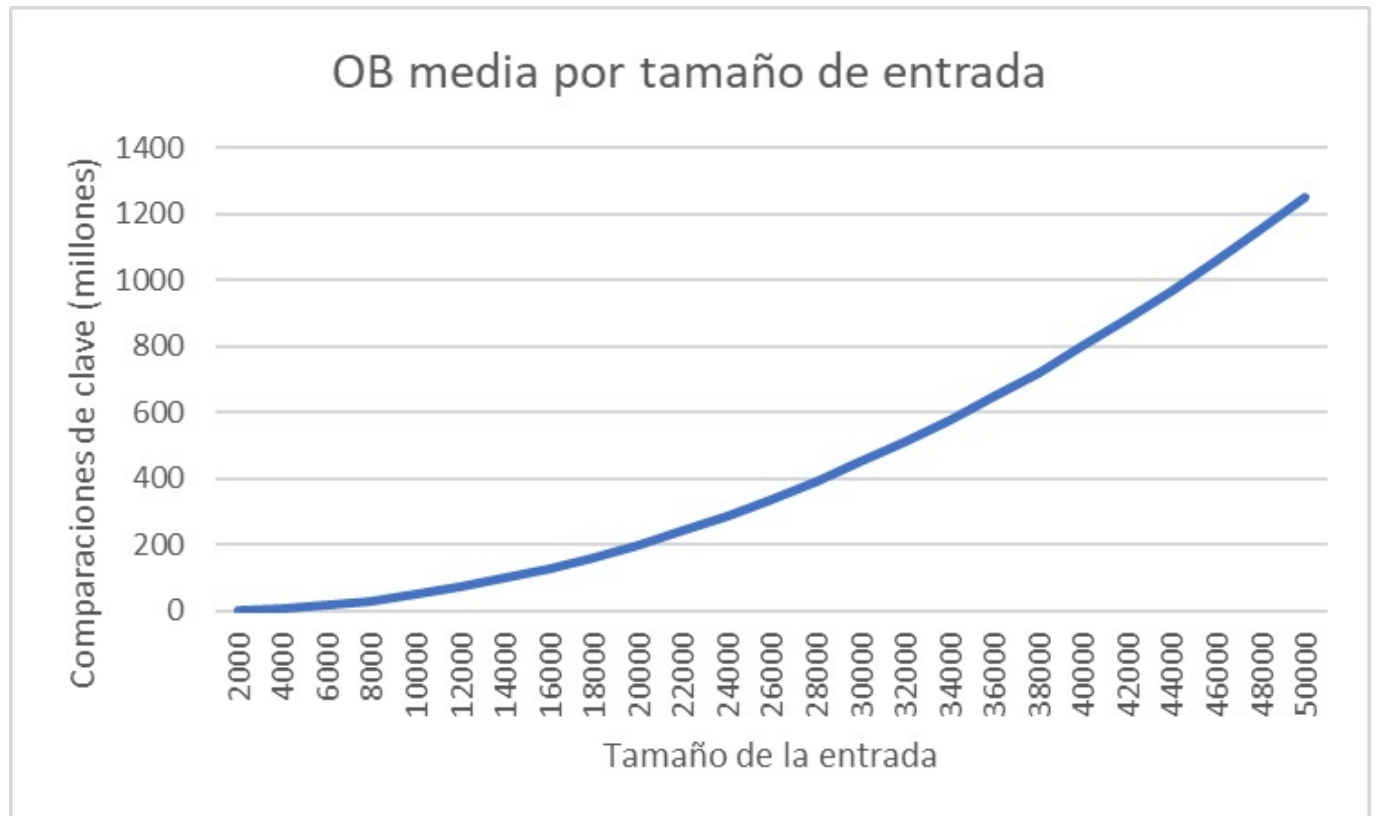
Comando ejecutado: `./exercise5 -num_min 2000 -num_max 50000 -incr 2000 -numP 50 -outputFile exercise5.log`

Lo primero que se observa es que el caso peor, mejor y medio eran iguales para todos los tamaños. Esto entra dentro de lo previsto pues, como se ha estudiado en clase, para SelectSort esto siempre se cumple.

La evolución del número de comparaciones necesarias sigue un crecimiento cuadrático, al igual que el tiempo medio, pues éste depende del número de comparaciones.

$$n_{\text{SelectSort}}(T, 1, N) = \frac{N^2}{2} - \frac{N}{2}$$

En las siguientes gráficas, se puede observar tanto el crecimiento del tiempo medio, como el del número de comparaciones (como se ha mencionado, la media de ob es igual al máximo y al mínimo):



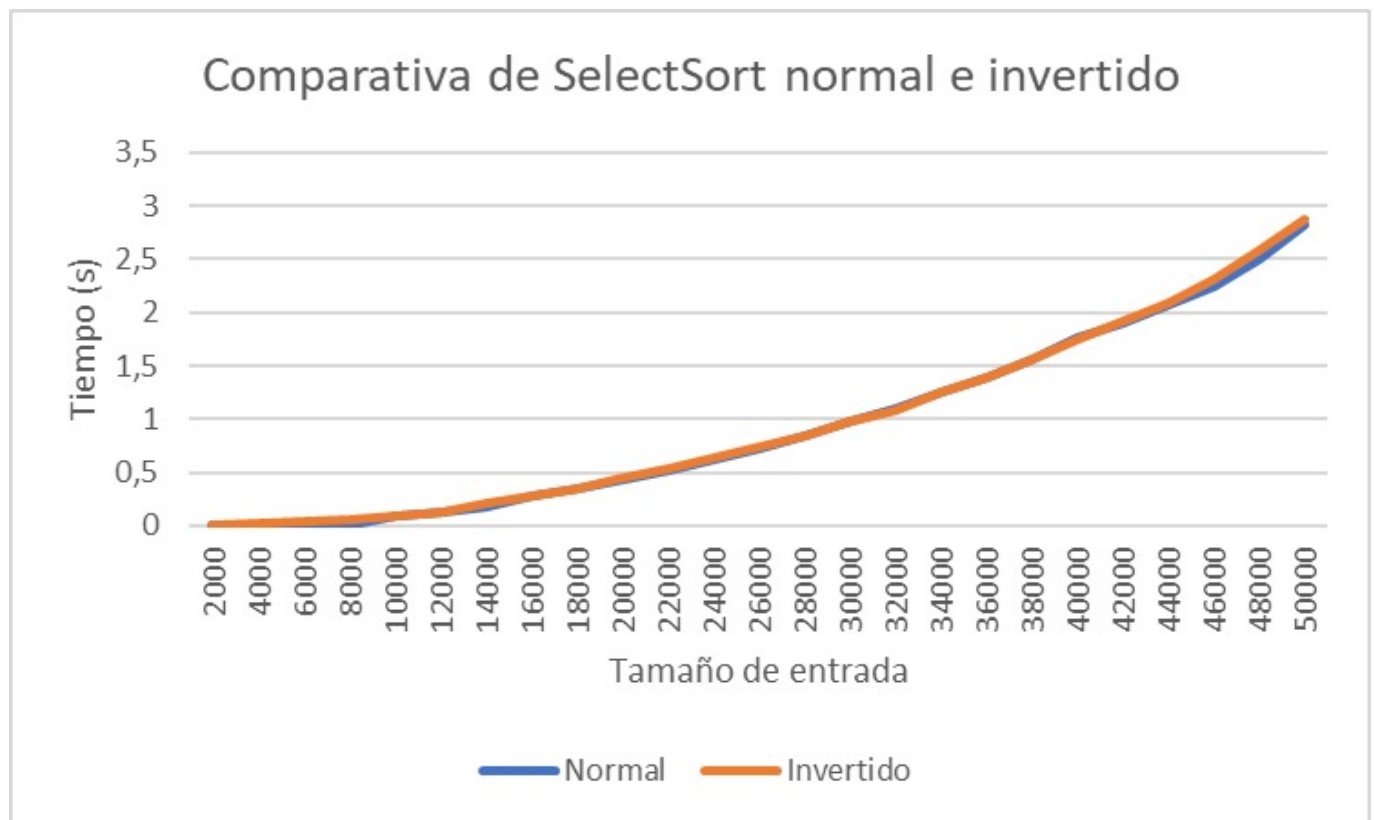
Como se puede apreciar, la curva del tiempo medio no es del todo suave, pues el tiempo de ejecución depende de parámetros externos, y muchos de ellos no se pueden controlar como se expone en el enunciado de la práctica.

Por otra parte, como el número de comparaciones es siempre el mismo en relación con el tamaño, esta curva describe perfectamente un crecimiento cuadrático.

5.6 Select Sort Invertido

Comando ejecutado: `./exercise5 -num_min 2000 -num_max 50000 -incr 2000 -numP 50 -outputFile exercise5.log`

Al ejecutar el ejercicio 5 con SelectSortInv, se observó que daba lugar a resultados casi idénticos a SelectSort, ya que tienen un funcionamiento muy parecido. En particular, el número de comparaciones para un mismo tamaño es el mismo. Por otra parte, el tiempo necesario para ejecutar ambas funciones prácticamente no difiere.





Como se ve en la imagen, los tiempos son prácticamente iguales. La diferencia entre ambas curvas es casi nula. Además la curva del número de comparaciones es exactamente igual a la de SelectSort, pues el funcionamiento de ambas funciones es esencialmente el mismo.

6. Respuestas a las preguntas teóricas

6.1 Pregunta 1

La función utilizada para generar números aleatorios se basa en el método expuesto en el capítulo 7 (Random Numbers) del libro "Numerical recipes in C: the art of scientific computing. En este texto, se muestra la siguiente sentencia para generar un número aleatorio entre 1 y un número cualquiera:

```
random=1+(int) (numero*rand()/(RAND_MAX+1.0));
```

Esta forma de calcular números primos funciona porque $\text{rand}()/(\text{RAND_MAX}+1) < 1$. Como se multiplica por el número máximo, tenemos que $(\text{int}) (\text{numero}*\text{rand}()/(\text{RAND_MAX}+1.0)) < \text{numero}$. El sumarle 1 permite que el número máximo también pueda aparecer.

Esta es una mejor forma de abordar el problema que esta otra:

```
random = 1+rand()%(numero);
```

Esto se debe a que la primera expresión reduce el sesgo. En general, el uso de la función módulo no es recomendable para la generación de números aleatorios, pues si $\text{RAND_MAX} + 1$ no es múltiplo del número, hay números con posibilidades ligeramente más altas de aparecer. Otra ventaja de la primera manera es la posibilidad de generar números aleatorios mayores que RAND_MAX .

Por estas dos razones, resulta mucho mejor usar la primera.

6.2 Pregunta 2

SelectSort tiene un funcionamiento muy sencillo. Simplemente en la iteración i coloca en la posición $i-1$ del array el elemento mínimo de entre los elementos que se encuentran en la subtabla formada por las celdas desde $i-1$ hasta la última. Como en la primera iteración la subtabla es la propia tabla completa, en la primera posición se coloca el mínimo. Después, el problema pasa a ser el mismo, pero con una tabla con una celda menos y sin el menor elemento. De esta forma, se puede asegurar que SelectSort ordena correctamente todos los elementos del array. Se trata de un algoritmo recursivo, que va reduciendo un problema a otro más sencillo, hasta acabar con el array.

6.3 Pregunta 3

No es necesario puesto que, por lo explicado antes, en la última posición solo se evaluará una subtabla de un elemento. Entonces, es innecesario hacer más comparaciones, pues la subtabla de un elemento está siempre ordenada.

6.4 Pregunta 4

En este caso, la operación básica es la comparación de claves. Esto lo sabemos porque esta instrucción se encuentra en el bucle más interno del programa, es decir, es la que más veces se realiza. Además, la comparación en la instrucción más representativa de SelectSort.

6.5 Pregunta 5

SelectSort es un algoritmo relativamente poco eficiente. Esto se debe a que dos arrays de N elementos necesitarán el mismo número de iteraciones para ser ordenadas, independientemente de en qué orden estén. El tiempo que tarda el algoritmo en ejecutarse está directamente relacionado con su tiempo abstracto de ejecución. Se tiene:

$$W_{SS}(N) = B_{SS} = \sum_{i=0}^{N-1} i = \frac{N(N-1)}{2} \sim \frac{N^2}{2} + O(N)$$

Esto se debe a que en la primera iteración se hacen $N-1$, y a partir de eso, en cada iteración, se hace una comparación menos que en la iteración anterior, hasta llegar a 1.

Pregunta 6

Teóricamente, **SelectSortInv** y **SelectSort** necesitan el mismo tiempo de ejecución, pues ambas son asintóticamente equivalentes a $N^2 + O(N)$. Esto se reafirma al observar la gráfica de comparación de ambas, esto se debe a que SelectSortInv tiene el mismo funcionamiento que SelectSort, solo que, en vez de poner en el primer puesto el mínimo, pone el máximo. Ambos algoritmos realizan el mismo número de comparaciones de clave, como vimos en el apartado 5.6.

7. Conclusiones finales

Después de haber estado trabajando con el algoritmo de ordenación local **Select Sort** podemos aseverar que no solo los algoritmos de ordenación locales de ordenación son poco eficaces por sus cotas inferiores: $n_{Alocal}(\sigma) \geq inv(\sigma)$, sino que además el algoritmo **Select Sort** solo es óptimo en el caso peor:

$$W_{SS}(N) \geq n_A([N, N-1, \dots, 2, 1]) \geq inv(n_A([N, N-1, \dots, 2, 1])) = \frac{N^2}{2} - \frac{N}{2}$$

Como en **Select Sort** tenemos que $W_{SS}(N) = A_{SS}(N) = B_{SS}(N)$, entonces no es óptimo en los casos mejor y medio lo que lo convierte en uno de los peores algoritmos de ordenación.